

# Automatic Test Pattern Generation Algorithms for Combinational and Sequential Circuits

Nhóm 4 – Khoa Điện-Điện tử

Tháng 10 năm 2026

# Vì sao cần testing và ATPG?

- Verification: thiết kế có đúng đặc tả? Testing: chip chế tạo có lỗi?
- Stuck-at: một dây bị kẹt 0 (SA0) hoặc 1 (SA1).
- ATPG tìm mẫu làm đáp ứng mạch tốt khác mạch lỗi tại ngõ ra.

## Hai điều kiện bắt buộc

Kích hoạt lỗi tại vị trí lỗi và lan truyền khác biệt đến ngõ ra quan sát được.

Nguồn: Wang, Wu, Wen (biên tập), 2006, Chương 1 và 4.

$$11 = \overline{3 \cdot 6}, \quad 16 = \overline{2 \cdot 11}, \quad 10 = \overline{1 \cdot 3}, \quad 22 = \overline{10 \cdot 16}.$$

- ❶ Đặt  $3 = 0$ : net 11 tốt bằng 1, lỗi bằng 0, tức  $D = (1, 0)$ .
- ❷ Đặt  $2 = 1$ : net  $16 = \overline{D} = (0, 1)$ ; net  $10 = 1$ .
- ❸ Ngõ ra  $22 = D$ : quan sát được lỗi.

Mẫu theo thứ tự PI (1, 2, 3, 6, 7)

$(X, 1, 0, X, X)$ ; ví dụ điền  $X = 0$  được 01000.

# Lộ trình và tiêu chí kiểm chứng

- P2–P3: D-algorithm và PODEM; cùng minh họa lỗi c17 11/SA0.
- P4: trạng thái, full scan và trải khung thời gian.
- P5–P6: cài đặt, trace, mô phỏng lỗi và coverage.
- Coverage phải ghi rõ danh sách lỗi; hết giới hạn tìm kiếm là `ABORTED`, không kết luận `UNTESTABLE`.

Video do P1 thực hiện, dài 2–3 phút

Hình 4.5, mục 4.3, trang 166–167:  $f = xy + \bar{y}z$ , tìm mẫu cho  $y$ /SA0.

# D-calculus: quan sát mạch tốt và mạch lỗi

| Giá trị        | Tốt | Lỗi |
|----------------|-----|-----|
| 0              | 0   | 0   |
| 1              | 1   | 1   |
| $X$            | ?   | ?   |
| $D$            | 1   | 0   |
| $\overline{D}$ | 0   | 1   |

## Tính cổng trên từng thành phần

$$D \wedge 1 = D, D \wedge \overline{D} = 0, D \wedge X = X.$$

Qua NAND với ngõ vào phụ bằng 1:  $D \leftrightarrow \overline{D}$ .

Bảng chuẩn tám cổng: `tests/data/five_valued_tables.md`.

# D-algorithm: chọn cube, lan truyền và justify

- NAND  $(a, b, z)$ : cover  $(0, X, 1)$ ,  $(X, 0, 1)$ ,  $(1, 1, 0)$ .
- PDCF cho  $z/SA0$ :  $(0, X, D)$  hoặc  $(X, 0, D)$ .
- PDC:  $(D, 1, \overline{D})$ ; ngõ vào phụ bằng 1 cho sai khác đi qua.
- Giao cube:  $X \cap v = v$ ; yêu cầu xác định trái nhau gây xung đột.
- D-frontier: đầu vào có sai khác, đầu ra còn  $X$ .
- J-frontier: đầu ra đã gán, đầu vào chưa justify được.

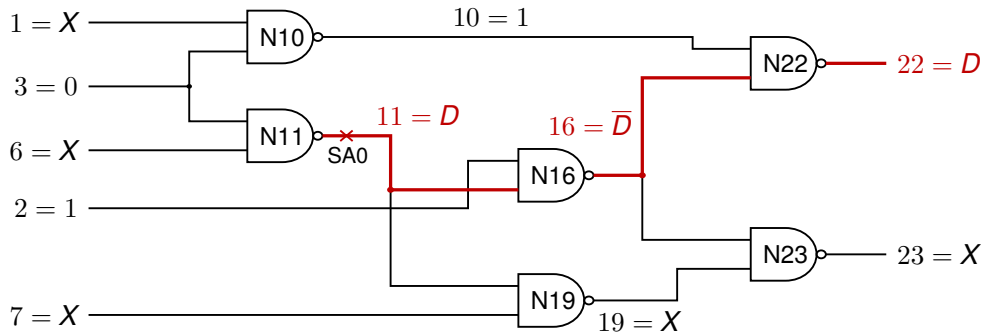
## Điều kiện thành công

Sai khác tới PO, mọi yêu cầu được justify và nhất quán.

Xung đột: quay lui, khôi phục trạng thái và thử cube khác.

Nguồn: J. Paul Roth, IBM J. Res. Dev., 1966, DOI: 10.1147/rd.104.0278.

## c17, 11/SA0: lan truyền và justification



- |   |                     |   |                                 |
|---|---------------------|---|---------------------------------|
| 1 | PDCF: 6 = 0, 11 = D | 2 | PDC: 2 = 1, 16 = $\bar{D}$      |
| 3 | PDC: 10 = 1, 22 = D | 4 | Justify: 3 = 0, J = $\emptyset$ |

Cube gốc (X, 1, 0, 0, X); hình sau tổng quát hóa (X, 1, 0, X, X).  
 X = 0: 01000; PO tốt (1, 1), lỗi (0, 0). 4 lần chọn cube, 0 backtrack.

# PODEM tìm vector bằng quyết định tại PI

- Objective xác định net và giá trị cần đạt.
- Backtrace đưa mục tiêu về một PI chưa gán.
- Imply mô phỏng tiến, kiểm tra lỗi tại PO.
- Nhánh thất bại: đảo PI; hết cả hai nhánh: quay về mức cha.

## Khác với D-algorithm

PODEM chỉ quyết định tại PI; net nội bộ được suy ra qua mô phỏng. Vẫn cần tìm kiếm và có thể gặp độ phức tạp hàm mũ.

Nguồn: Goel (1981); Wang, Wu và Wen (2006), mục 4.4.4.

## c17: hai quyết định phát hiện 11/SA0

| Bước | Objective | Gán PI  | Kết quả chính               |
|------|-----------|---------|-----------------------------|
| 1    | (11, 1)   | $3 = 0$ | $11 = D, 10 = 1$            |
| 2    | (2, 1)    | $2 = 1$ | $16 = \overline{D}, 22 = D$ |

### Kết quả theo thứ tự PI (1, 2, 3, 6, 7)

Pattern  $X10XX$ ; 0 backtrack. Điền X bằng 0: 01000.

- Cả 8 cách điền X đều phát hiện lỗi tại PO22.
- Frontier cuối vẫn là  $\{19, 23\}$ ; có  $D$  tại PO là đủ dừng.

# Một lần quay lui dẫn tới vector 01

$$t = a + b, \quad n = \bar{a}, \quad out = t \cdot n; \quad t/SA0.$$

- ❶ Thử  $a = 1$ :  $t = D$ , nhưng  $n = 0$  làm  $out = 0$ .
- ❷ Đảo  $a = 0$ :  $n = 1$ ,  $t = X$ ; tiếp tục kích hoạt.
- ❸ Gán  $b = 1$ :  $t = D$ ,  $out = D$ ; phát hiện lỗi.

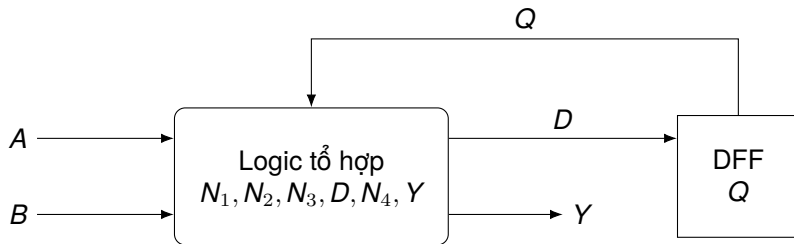
## Điểm kiểm tra code P5

3 hàng gán/đảo, 1 backtrack; pattern  $(a, b) = 01$ . Giới hạn 0 phải trả ABORTED, không phải UNTESTABLE.

P2: 4 lần chọn cube, 0 backtrack; P3: 2 phép gán PI trên c17. Khác đơn vị đếm, chưa suy ra tốc độ. Chờ trace P5 và simulator P6.

## P4 – Vì sao ATPG tuần tự khó?

- DFF lưu trạng thái:  $Q(t+1) = D(t)$ ;  $Q(0)$  có thể chưa biết.
- Không scan: khó điều khiển  $Q$ , khó quan sát  $D$ ; cần chuỗi PI qua nhiều chu kỳ để khởi tạo, kích hoạt và lan truyền lỗi.
- Mạch ví dụ P4: 2 PI, 1 PO, 6 cổng tổ hợp, 1 DFF.



## P4 – Trả khung và ví dụ $N_3/SA0$

- Thay DFF bằng ràng buộc  $Q_{t+1} = D_t$ ; cùng một lỗi hiện diện tại mỗi khung của mạch đã trả.
- Dùng hai khung:  $(A_0, B_0) = (1, 0)$ ,  $(A_1, B_1) = (1, 0)$ .

| Khung | $N_3$ tốt/lỗi | $D$ tốt/lỗi | $Y$ tốt/lỗi |
|-------|---------------|-------------|-------------|
| 0     | 1/0           | 0/1         | $Q_0/Q_0$   |
| 1     | 1/0           | 0/1         | 0/1         |

Lỗi quan sát tại PO ở khung 1, với cả  $Q_0 = 0$  và  $Q_0 = 1$ .

## P4 – Full scan và kết quả kiểm chứng

- Scan FF: shift in  $Q \rightarrow$  capture  $D \rightarrow$  shift out. ATPG tổ hợp xem  $Q$  là pseudo-PI,  $D$  là pseudo-PO.
- Với  $N_3/SA0$ :  $A = 1, B = 0, Q = 0$  cho  $D = 0/1$ .

| Mô hình (18 stem SA)    | Phát hiện bảo đảm |
|-------------------------|-------------------|
| Full scan, bộ mẫu PODEM | 18/18             |
| Không scan, $k = 1$     | 1/18              |
| Không scan, $k = 2$     | 13/18             |
| Không scan, $k = 3$     | 18/18             |

Full scan: PODEM và simulator, coverage của cả bộ mẫu. Không scan:  $Q_0$  chưa biết; PODEM + vét cạn chuỗi bổ sung. Chi tiết và nguồn từng hàng trong `results/seq_integrated_*.md`.

- Triển khai PODEM trên logic 5 giá trị.
- Các bước chính:
  - Imply
  - Objective
  - Backtrace
  - Gán PI
  - Backtrack
- Sử dụng D-frontier để lan truyền fault.
- Bổ sung xử lý Objective và Backtrace cho XOR/XNOR.

Objective  $\rightarrow$  Backtrace  $\rightarrow$  PI Assignment  $\rightarrow$  Imply

- Thử nhánh đầu tiên.
- Khi nhánh thất bại, đảo quyết định PI.
- Sau khi backtrack, implication lại trạng thái mới.
- Tiếp tục tìm kiếm với decision stack.
- Phân biệt UNTESTABLE và ABORTED.

| Fault            | Pattern | Backtracks | Status   |
|------------------|---------|------------|----------|
| c17: 11/SA0      | X10XX   | 0          | DETECTED |
| Backtrack: t/SA0 | 01      | 1          | DETECTED |

- Hỗ trợ fault-frame và branch fault.
- Kiểm thử XOR/XNOR trên logic 5 giá trị.

334 tests passed (tích hợp)

## P6 – Kiểm chứng độc lập

- Đọc `.bench` → fault list → PODEM (P5) → mô phỏng lỗi độc lập → coverage, nén tập test.
- c17: 34 lỗi gốc (22 stem + 12 nhánh), gộp tương đương còn 22 đại diện.
- Mỗi pattern được mô phỏng lại ở mạch tốt và mạch lỗi; pattern có `X` chỉ được xác nhận khi đã vét cạn mọi cách điền.
- Đầu vào sai (SA khác 0/1, nhánh không tồn tại,  $k \leq 0$ ) bị từ chối.
- Mạch tuần tự: `Q@0` chưa biết; chỉ chuỗi bảo đảm với mọi trạng thái đầu mới tính là phát hiện.

## P6 – Kết quả c17 và mạch tuần tự

| c17, PODEM              | 22 đại diện | 34 lỗi gốc |
|-------------------------|-------------|------------|
| Fault coverage          | 22/22       | 34/34      |
| Backtrack trung bình    | 0           | 0          |
| Pattern trước / sau nén | 22 / 6      | 34 / 7     |
| Pattern kiểm chứng sai  | 0           | 0          |

| seq_example, 18 lỗi                  | $k = 1$ | $k = 2$ | $k = 3$ |
|--------------------------------------|---------|---------|---------|
| Phát hiện bảo đảm ( $Q@0$ chưa biết) | 1/18    | 13/18   | 18/18   |

Full scan (P4): 18/18. Tập 6 pattern phủ cả 34 lỗi gốc; tuần tự: PODEM + vét cạn chuỗi bổ sung, khớp mốc độc lập P4.

- **Trace một lỗi:** `python -m atpg.run circuits/c17.bench --fault 11 0 --trace`
- **Backtrack:** `python -m atpg.run circuits/backtrack_example.bench --fault t 0 --trace`
- **Toàn bộ lỗi:** `python -m atpg.run circuits/c17.bench --all`
- **Tuần tự:** `python -m atpg.run circuits/seq_example.bench --unroll 2 --all`

Cảm ơn thầy và các bạn đã lắng nghe. Xin mời đặt câu hỏi.